PRAKTIKUM PEMROGRAMAN BERORIENTASI OBJEK

ABSTRACTION



Disusun Oleh:

Haliya Isma Husna Putri Ahmadi

2511532002

Dosen Pengampu:

Afdhal Dinilhak. .M.Kom,.

Asisten Praktikum:

Ezza Andini

PROGRAM STUDI INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

A. IDENTITAS

Nama : Haliya Isma Husna Putri Ahmadi
NIM : 2511532002
Kelas : B

Output tools.NimParams:

```
Parameter tugas untuk NIM 2511532002
  CARD_ADMIN_PERCENT  = 3
  TRANSFER_ADMIN_FEE  = 3500
  SMS_MAX_LENGTH      = 102
```

B. PENJELASN KODE PROGRAM

Berikut merupakan penjelasan kode program dari Praktikum pekan-6 Mata Kuliah Praktikum Pemrograman Berorientasi Objek (PBO) yang mencakup *Abstraction* menggunakan bahasa *Java.*

a) Latihan

1. L1 - Abstract class dan abstract method

Pada latihan L1, dibuat sebuah sistem toko online UMKM sederhana yang menerapkan konsep abstraction. Sistem dirancang untuk mengelola berbagai jenis produk yang dijual serta menghitung biaya pengiriman berdasarkan berat barang.

Sistem menggunakan abstract class Product untuk detail setiap product nantinya (sebagai kelas induk) dan class abstract class ShippingService untuk implementasi biaya pengirmanan di sistem.

No.	Nama Class	Jenis	Fungsi
1	*Product*	*Abstract class*	Kelas induk untuk seluruh produk.
2	*PhysicalProduct*	*Class*	Mengelola untuk *physical product*
3	*DigitalProduct*	*Class*	Mengelola untuk *digital product*

4	*ShippingService*	*Abstract class*	Kelas induk
5	*RegularShipping*	*Class*	
6	*ExpressShipping*	*Class*	Mengatur tarif pengiriman ekspres.

a. *Product (src/shop/model/Product.java)*

Class *product* merupakan *abstract class* yang menyediakan strukturobjek produk secara umum.

package shop.model;

```
public abstract class Product {
    private final String sku;
    private final String name;
    private final long price; //in rupiah

    protected Product(String sku, String name, long price) {
        if (sku == null || sku.isBlank()) {
            throw new IllegalArgumentException("SKU cannot be null or blank");
        }
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Name cannot be null or blank");
        }
        if (price < 0) {
            throw new IllegalArgumentException("Price cannot be negative");
        }
        this.sku = sku;
        this.name = name;
        this.price = price;
    }
public String getSku(){return sku;}
public String getName(){return name;}
public long getPrice(){return price;}

/**Price the customer actually pays. Subclasses may override it. */
public long getFinalPrice() {
```

```
    return price;
}
/** Every concrete product must state its category. */
public abstract String category();

/** A concrete method that relies on the abstract methods category(). */
public String getInfo() {
    return "["+category() + "] "+getName()+" - Rp"+getFinalPrice();
}
}
```

Penjelasan kode:

1) *public abstract class Product*

   Digunakan untuk mendeklarasikan *Product* sebagai *abstract class* sehingga tidak dapat dibuat menjadi objek secara langsung.

2) *Atribut sku, name, dan price*

   Digunakan untuk menyimpan kode produk, nama produk, dan harga. Ketiganya menggunakan *private final* agar tidak dapat diakses langsung dari luar *class* dan nilainya tidak dapat diubah setelah objek dibuat.

3) *Konstruktor Product()*

   Digunakan untuk menginisialisasi atribut produk. Terdapat validasi agar SKU dan nama tidak kosong serta harga tidak bernilai negatif.

4) *Method getSku(), getName(), dan getPrice()*

   Merupakan *getter* yang digunakan untuk mengambil nilai atribut.

5) *Method getFinalPrice()*

   Digunakan untuk mengembalikan harga akhir produk. Pada tahap ini, harga akhir masih sama dengan harga awal.

6) Method category()

   Merupakan *abstract method* yang tidak memiliki implementasi dan wajib dibuat ulang oleh setiap *subclass*.

7) *Method getInfo()*

Digiunakan untuk menampilkan informasi produk berupa kategori, nama, dan harga akhir.

b. *PhysicalProduct (src/shop/model/PhysicalProduct.java)*

Class PhysicalProduct merupakan *subclass Product* yang digunakan untuk merepresentasikan produk berbentuk fisik atau yang dapat dikirimkan kepada pelanggan, seperti kopi dan rendang kemasan.

```
package shop.model;
import shop.shipping.Shippable;

public class PhysicalProduct extends Product implements Shippable {
   private final int weightGrams;

      public PhysicalProduct(String sku, String name, long price, int weightGrams){
      super(sku, name, price);
      if (weightGrams <= 0) {
            throw new IllegalArgumentException("Weight cannot be zero or negative");
      }
      this.weightGrams = weightGrams;
   }
   @Override
   public String category() {
      return "Fisik";
   }
}
```

Penjelasan kode:

1) *extends Product*

   Menunjukkan *PhysicalProduct* merupakan *subclass* yang mewarisi atribut dan method dari *class Product*.
2) Atribut *weightGrams* digunakan untuk menyimpan berat produk dalam satuan gram.
3) Konstruktor *PhysicalProduct*() menerima parameter *SKU*, nama, harga, dan berat produk.
4) *super(sku, name, price)* digunakan untuk memanggil konstruktor *class* induk *Product* sehingga atribut umum produk dapat diinisialisasi.
5) Validasi *weightGrams <= 0* digunakan untuk memastikan berat produk harus lebih besar dari nol. Jika tidak sesuai, program akan menghasilkan *IllegalArgumentException*.
6) *@Override* menunjukkan bahwa *method category()* mengimplementasikan *abstract* method dari *class Product* dengan mengembalikan nilai "Fisik".

c. *DigitalProduct (src/shop/model/DigitalProduct.java)*

Class DigitalProduct merupakan *subclass* dari *Product* yang digunakan untuk merepresentasikan produk digital, seperti *e-book* atau produk yang dapat diunduh oleh pelanggan.

```
package shop.model;
public class DigitalProduct extends Product {

  private final String downloadUrl;

  public DigitalProduct(String sku, String name,
                        long price, String downloadUrl) {
    super(sku, name, price);

    if (downloadUrl == null || downloadUrl.isBlank()) {
      throw new IllegalArgumentException(
```

```
                "Download URL must not be blank");
        }
        this.downloadUrl = downloadUrl;
    }
    public String getDownloadUrl() {
        return downloadUrl;
    }
    @Override
    public String category() {
        return "Digital";
    }
}
```

Penjelasan Kode:

1) *extends Product* menunjukkan bahwa *DigitalProduct* mewarisi struktur dan method dari *class Product*.
2) *Atribut downloadUrl* digunakan untuk menyimpan alamat tautan yang digunakan pelanggan untuk mengunduh produk digital.
3) Konstruktor *DigitalProduct*() menerima parameter *SKU*, nama, harga, dan URL unduhan.
4) *super(sku, name, price)* digunakan untuk memanggil konstruktor *class* induk *Product*.
5) Validasi *downloadUrl* memastikan alamat unduhan tidak bernilai null atau kosong.
6) *Method getDownloadUrl*() digunakan untuk mengambil alamat unduhan produk.
7) *Method category*() mengimplementasikan *abstract method* dari *Product* dengan mengembalikan nilai "Digital".

d. *ShippingService (src/shop/shipping/ShippingService.java)*

Class ShippingService merupakan *abstract class* yang digunakan sebagai kelas induk untuk berbagai jenis layanan pengiriman.

```
package shop.shipping;
public abstract class ShippingService {

    private final String name;
    protected ShippingService(String name) {
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException(
                    "Name must not be blank");
        }
        this.name = name;
    }
    public String getName() {
        return name;
    }
    protected abstract long ratePerKg();
    public long calculateCost(int weightGrams) {
        if (weightGrams <= 0) {
            return 0;
        }
        long kilograms = (weightGrams + 999L) / 1000;
        return kilograms * ratePerKg();
    }
}
```

Penjelasan Kode:

1) *public abstract class ShippingService*
   Digunakan untuk mendeklarasikan kelas induk layanan pengiriman yang tidak dapat dibuat menjadi objek secara langsung.
2) *Atribut name* digunakan untuk menyimpan nama layanan pengiriman dan bersifat *private final*.
3) Konstruktor *ShippingService()* digunakan untuk menginisialisasi nama layanan pengiriman serta melakukan validasi agar nama tidak kosong.

4) *Method getName()*

   Digunakan untuk mengambil nama layanan pengiriman.

5) *Method ratePerKg()*

   Merupakan *abstract method* yang digunakan untuk menentukan tarif pengiriman per kilogram. Implementasinya diserahkan kepada *subclass*.

6) Method calculateCost()

   Digunakan untuk menghitung biaya pengiriman berdasarkan berat barang.

7) Pada proses perhitungan, berat barang dibulatkan ke atas berdasarkan satuan kilogram menggunakan rumus (*weightGrams* + 999L) / 1000. Hasilnya kemudian dikalikan dengan tarif per kilogram dari *method ratePerKg()*

e. *RegularShipping (src/shop/shipping/RegularShipping.java)*

Class RegularShipping merupakan subclass dari *ShippingService* yang digunakan untuk mengimplementasikan layanan pengiriman reguler dengan tarif Rp10.000 per kilogram.

```
package shop.shipping;
public class RegularShipping extends ShippingService {
  public RegularShipping() {
     super("Reguler");
  }

  @Override
  protected long ratePerKg() {
     return 10_000;
  }
}
```

Penjelasn Kode:

1) *extends ShippingService*
   Menunjukkan bahwa *RegularShipping* mewarisi struktur dan method dari class *ShippingService*.
2) Konstruktor *RegularShipping*() menggunakan *super*("*Reguler*") untuk memberikan nama layanan pengiriman kepada class induk.
3) *Method ratePerKg()* mengimplementasikan *abstract method* dari *ShippingService* dengan mengembalikan tarif sebesar Rp10.000 per kilogram.
4) *@Override*
   Digunakan untuk menandakan bahwa *method* tersebut merupakan implementasi ulang dari *method* milik *class* induk.

f. *ExpressShipping (src/shop/shipping/ExpressShipping.java)*

Class ExpressShipping merupakan *subclass* dari *ShippingService* yang digunakan untuk mengimplementasikan layanan pengiriman ekspres dengan tarif Rp18.000 per kilogram.

```
package shop.shipping;
public class ExpressShipping extends ShippingService {

   public ExpressShipping() {
      super("Ekspres");
   }
   @Override
   protected long ratePerKg() {
      return 18_000;
   }
}
```

Penjelasn kode:

1) *extends ShippingService* menunjukkan *ExpressShipping* merupakan turunan dari class *ShippingService*.
2) Konstruktor *ExpressShipping()* menggunakan *super("Ekspres")* untuk memberikan nama layanan pengiriman ekspres.
3) *Method ratePerKg()*
   Mengimplementasikan *abstract method* dari *class* induk dengan mengembalikan tarif sebesar Rp18.000 per kilogram.
4) *Method calculateCost*() tidak perlu dibuat ulang karena sudah tersedia pada class *ShippingService* dan dapat digunakan oleh *ExpressShipping*.

2. L2 - Interface dan program to an interface

Pada L2, sistem toko online UMKM dikembangkan pengiriman melalui *interface Shippable* dan kemampuan pembayaran melalui *interface PaymentMethod*. Sistem juga dikembangkan dengan *class CheckoutService* untuk menghitung subtotal, berat produk, total belanja, serta memproses pembayaran.

No.	Nama Class	Jenis	Fungsi
1	*Shippable*	*Interface*	*Interface* untuk produk yang dapat dikirim.
2	*PhysicalProduct*	*Class*	Diubah agar mengimplementasikan interface *Shippable*.
3	*PaymentMethod*	*Interface*	*Interface* yang menjadi kontrak metode pembayaran.
4	*EWalletPayment*	*Class*	Pembayaran menggunakan *e-wallet*

5	*BankTransferPayment*	*Class*	Pembayaran melalui transfer bank.
6	*CheckoutService*	*Class*	Class untuk menghitung total belanja dan memproses pembayaran.
7	*ShopApp*	*Class*	Class utama untuk menjalankan dan menguji sistem toko.

a. Shippable (shop/shipping/Shippable.java)

Interface Shippable dibuat untuk mendefinisikan kemampuan produk yang dapat dikirim. *Interface* ini memiliki *method getWeightGrams()* yang digunakan untuk memperoleh berat produk dalam satuan gram.

```
package shop.shipping;

public interface Shippable {
    int getWeightGrams();
}
```

b. PhysicalProduct (shop/model/PhysicalProduct.java)

PhysicalProduct mengalami perubahan dengan menambahkan implements *Shippable*. *Method getWeightGrams*() kemudian dibuat untuk mengembalikan berat produk.

```
public class PhysicalProduct extends Product implements Shippable {
    private final int weightGrams;

    public PhysicalProduct(String sku, String name,
                           long price, int weightGrams) {
        super(sku, name, price);
        if (weightGrams <= 0) {
            throw new IllegalArgumentException(
```

```
            "Weight must be positive");
        }
        this.weightGrams = weightGrams;
    }
    @Override
    public int getWeightGrams() {
        return weightGrams;
    }
    @Override
    public String category() {
        return "Fisik";    }}
```

c. *PaymentMethod (shop/payment/PaymentMethod.java)*

Interface PaymentMethod dibuat untuk seluruh metode pembayaran. *Interface* ini memiliki dua *method*, yaitu *getName()* untuk memperoleh nama metode pembayaran dan *pay(long amount*) untuk memproses pembayaran.

```
package shop.payment;

public interface PaymentMethod {
    String getName();

    boolean pay(long amount);
}
```

d. *EwalletPayment (shop/payment/EWalletPayment.java)*

Class EWalletPayment dibuat dengan mengimplementasikan *interface PaymentMethod*.

```
package shop.payment;


public class EWalletPayment implements PaymentMethod {
    private final String provider;
    private long balance;

    public EWalletPayment(String provider, long balance) {
        if (provider == null || provider.isBlank()) {
            throw new IllegalArgumentException(
```

```
            "Provider must not be blank");
    }
    if (balance < 0) {
        throw new IllegalArgumentException(
            "Balance must not be negative");
    }
    this.provider = provider;
    this.balance = balance;
}
@Override
public String getName() {
    return "E-Wallet " + provider;
}
public long getBalance() {
    return balance;
}

@Override
public boolean pay(long amount) {
    if (amount <= 0 || amount > balance) {
        return false;
    }
    balance -= amount;
    return true;  }}
```

Penjelasan kode:

1) Atribut *provider* dan *balance* untuk menyimpan nama penyedia layanan serta saldo pengguna.
2) *Method pay*() digunakan untuk memproses pembayaran. Jika jumlah pembayaran tidak valid atau melebihi saldo, pembayaran akan gagal. Jika berhasil, saldo akan dikurangi sesuai jumlah pembayaran.

e. BankTransferPayment(shop/payment/BankTransferPayment.java)

Class BankTransferPayment digunakan untuk merepresentasikan pembayaran melalui transfer bank. *Class* ini mengimplementasikan *interface PaymentMethod*, dengan *method getName*() untuk menampilkan nama *bank* dan *pay*() untuk melakukan simulasi pembayaran. Pembayaran dinyatakan berhasil apabila jumlah pembayaran lebih dari nol.

```
package shop.payment;
public class BankTransferPayment implements PaymentMethod {
   private final String bankName;

   public BankTransferPayment(String bankName) {
      this.bankName = bankName;
   }
   @Override
   public String getName() {
      return "Transfer " + bankName;
   }
   @Override
   public boolean pay(long amount) {
      return amount > 0;}}
```

f. CheckoutService (shop/service/CheckoutService.java)

Class CheckoutService digunakan untuk mengelola proses transaksi.

```
package shop.service;

import java.util.List;
...

public class CheckoutService {
```

```
public long subtotal(List<Product> items) {
    long total = 0;
    for (Product product : items) {
        total += product.getFinalPrice();
    }
    return total;
}

public int totalWeight(List<Product> items) {
    int grams = 0;
    for (Product product : items) {
        if (product instanceof Shippable shippable) {
            grams += shippable.getWeightGrams();
        } }
    return grams;
}

public long total(List<Product> items,
                  ShippingService shipping) {
    return subtotal(items)
        + shipping.calculateCost(totalWeight(items));
}

public boolean checkout(List<Product> items,
                        ShippingService shipping,
                        PaymentMethod payment) {
    if (items.isEmpty()) {
        return false;
    }

    return payment.pay(total(items, shipping)); }}
```

Penjelasn kode:

1) *Method subtotal()* menghitung total harga produk.
2) *totalWeight*() menghitung berat seluruh produk yang dapat dikirim dengan memeriksa *interface Shippable*.
3) *Method total*() menghitung total belanja beserta biaya pengiriman.
4) *Method checkout()* memproses pembayaran menggunakan *interface PaymentMethod*. Dengan demikian, sistem tidak bergantung pada *class* pembayaran tertentu dan dapat menerima berbagai metode pembayaran.

g. ShopApp (shop/app/ShopApp.java)

Pada *ShopApp*, program total Reguler masih *265000 dan e-book* masih *50000*, digunakan sebagai pengujian untuk menghitung subtotal, berat produk, dan total belanja berdasarkan layanan pengiriman.

3. L3 - Interface lanjutan: default, static, private method dan interface inheritance

Pada L3, sistem toko online UMKM dikembangkan dengan menambahkan fitur diskon pada produk digital, fitur *refund* pada metode pembayaran, dan perhitungan cicilan kartu kredit. Dengan menambhakn *interface*, yaitu *default method, static method*, *private method*, serta pewarisan *interface* (interface inheritance).

No.	Nama Class	Jenis	Fungsi
1	*Discountable*	*Interface*	*Interface* untuk mengatur pemberian diskon produk.

2	*DigitalProduct*	*Class*	Dimodifikasi agar mengimplementasikan *Discountable*.
3	*RefundablePayment*	*Interface*	*Interface* turunan *PaymentMethod (*mendukung *refund).*
4	*InstallmentPayment*	*Class*	*Interface* turunan *PaymentMethod (cicilan),*
5	*EWalletPayment*	*Class*	Dimodifikasi agar mendukung fitur *refund*.
6	*CreditCardPayment*	*Class*	Class pembayaran kartu kredit dengan fitur *refund* dan cicilan.
7	*CheckoutService*	*Class*	Ditambahkan method untuk memproses refund
8	*ShopApp*	*Class*	Dilengkapi dengan pengujian pembayaran, cicilan, dan refund.

a. Discountable (shop/discount/Discountable.java)

Interface Discountable dibuat untuk mengatur mekanisme diskon pada produk.

```
package shop.discount;
public interface Discountable {

    int discountPercent();
    default long applyDiscount(long price) {
        return price - percentOf(price, safePercent()); }
    static long percentOf(long amount, int percent) {
        return amount * percent / 100;
    }
```

```
    private int safePercent() {
        int percent = discountPercent();
        if (percent < 0) {
            return 0;
        }
        return Math.min(percent, 100);  }}
```

Penjelasan Kode:

1) *default method applyDiscount()* untuk menghitung harga setelah diskon
2) *static method percentOf()* untuk menghitung persentase potongan.
3) *pivate method safePercent()* untuk memastikan persentase diskon berada pada rentang 0–100%.

b. DigitalProduct (shop/model/DigitalProduct.java)

Pada bagian ini, *DigitalProduct* mengalami perubahan dengan menambahkan *implements Discountable*. *Method discountPercent()* digunakan untuk menentukan diskon sebesar 10%, sedangkan *getFinalPrice() di-override* agar mengembalikan harga produk setelah diskon diterapkan.

```
package shop.model;
import shop.discount.Discountable;

public class DigitalProduct extends Product implements Discountable {
    private static final int DIGITAL_DISCOUNT_PERCENT = 10;
    private final String downloadUrl;

    public DigitalProduct(String sku, String name,
                          long price, String downloadUrl) {
        super(sku, name, price);

        if (downloadUrl == null || downloadUrl.isBlank()) {
```

```
            throw new IllegalArgumentException(
                "Download URL must not be blank");  }

        this.downloadUrl = downloadUrl;   }

    public String getDownloadUrl() {
        return downloadUrl;}

    @Override
    public String category() {
        return "Digital";
    }
    @Override
    public int discountPercent() {
        return DIGITAL_DISCOUNT_PERCENT;
    }
    @Override
    public long getFinalPrice() {
        return applyDiscount(getPrice());}}
```

Dengan demikian, harga *e-book* yang sebelumnya Rp50.000 menjadi Rp45.000. *CheckoutServic*e tidak perlu diubah karena *method subtotal()* sudah menggunakan *getFinalPrice()*.

c. *RefundablePayment (shop/payment/RefundablePayment.java)*

Interface RefundablePayment merupakan turunan dari PaymentMethod. Interface ini menambahkan *method refund()* untuk mendefinisikan kemampuan pengembalian dana pada metode pembayaran tertentu.

```
package shop.payment;

public interface RefundablePayment extends PaymentMethod {
    boolean refund(long amount);
}
```

d. *InstallmentPayment (shop/payment/InstallmentPayment.java)*

Interface InstallmentPayment digunakan untuk mendefinisikan kemampuan pembayaran secara cicilan. Method *maxInstallmentMonths()* menentukan batas maksimal waktu cicilan, sedangkan *monthlyInstallment()* menghitung nominal cicilan per bulan dengan pembulatan ke atas. *Method* tersebut juga memeriksa agar jumlah bulan tidak kurang dari satu atau melebihi batas yang ditentukan.

```
package shop.payment;
public interface InstallmentPayment extends PaymentMethod {
   int maxInstallmentMonths();

   default long monthlyInstallment(long amount, int months) {
      if (months < 1 || months > maxInstallmentMonths()) {
         throw new IllegalArgumentException(
            "Months must be between 1 and "
            + maxInstallmentMonths());}
      return (amount + months - 1) / months;  }}
```

e. *EwalletPayment (shop/payment/EWalletPayment.java)*

Pada L3, deklarasi *class* diubah dari *implements PaymentMethod* menjadi *implements RefundablePayment*. *Method refund()* juga ditambahkan untuk mengembalikan *saldo e-wallet.*

```
package shop.payment;

public class EWalletPayment implements RefundablePayment {
   private final String provider;
   private long balance;

   public EWalletPayment(String provider, long balance) {
      if (provider == null || provider.isBlank()) {
         throw new IllegalArgumentException(
            "Provider must not be blank");  }
      if (balance < 0) {
         throw new IllegalArgumentException(
            "Balance must not be negative"); }
```

```
    this.provider = provider;
    this.balance = balance;    }
@Override
public String getName() {
    return "E-Wallet " + provider;
}
public long getBalance() {
    return balance;
}
@Override
public boolean pay(long amount) {
    if (amount <= 0 || amount > balance) {
        return false;}
    balance -= amount;
    return true;   }
@Override
public boolean refund(long amount) {
    if (amount <= 0) {
        return false;
    }
    balance += amount;
    return true;  }}
```

Penjelasan Kode:

Method refund() digunakan untuk mengembalikan dana ke *saldo e-wallet*. Jika jumlah refund kurang dari atau sama dengan nol, proses akan gagal. Jika jumlahnya valid, saldo akan bertambah sesuai nominal pengembalian.

f. CreditCardPayment (shop/payment/CreditCardPayment.java)

Class CreditCardPayment dibuat dengan mengimplementasikan dua *interface* sekaligus, yaitu *RefundablePayment* dan *InstallmentPayment*. Class ini memiliki atribut *cardHolder*, *creditLimit*, dan *usedCredit* untuk menyimpan nama pemegang kartu, batas kredit, serta jumlah kredit yang telah digunakan.

```
package shop.payment;
```

```
public class CreditCardPayment implements RefundablePayment,
InstallmentPayment {
    private final String cardHolder;
    private final long creditLimit;
    private long usedCredit;

    public CreditCardPayment(String cardHolder, long creditLimit) {
        this.cardHolder = cardHolder;
        this.creditLimit = creditLimit;
        this.usedCredit = 0;    }

    @Override
    public String getName() {
        return "Kartu Kredit (" + cardHolder + ")";    }

    @Override
    public boolean pay(long amount) {
        if (amount <= 0 || usedCredit + amount > creditLimit) {
            return false;     }

        usedCredit += amount;
        return true;    }

    @Override
    public boolean refund(long amount) {
        if (amount <= 0 || amount > usedCredit) {
            return false;     }

        usedCredit -= amount;
        return true;    }

    @Override
    public int maxInstallmentMonths() {
        return 12;  }

    public long getUsedCredit() {
```

return usedCredit; }}

Penjelasan Kode;

1) *Method pay()* digunakan untuk memproses pembayaran selama nominalnya valid dan tidak melebihi batas kredit.
2) *Method refund()* digunakan untuk mengurangi kredit yang telah digunakan ketika pengembalian dana berhasil.
3) *maxInstallmentMonths()* menentukan batas cicilan maksimal selama 12 bulan.

g. CheckoutService (src\shop\service\ CheckoutService.java)

Pada *CheckoutService*, ditambahkan *method refund*() untuk meneruskan proses pengembalian dana ke objek pembayaran yang mendukung *interface RefundablePayment*.

```
public boolean refund(RefundablePayment payment, long amount) {
   return payment.refund(amount);
}
```

Method tersebut menerima parameter bertipe *RefundablePayment,* dengan demikian, hanya metode pembayaran yang mengimplementasikan *interface* tersebut yang dapat digunakan untuk melakukan *refund*.

h. ShopApp (shop/app/ShopApp.java)

Pada *ShopApp*, ditambahkan pengujian untuk metode pembayaran *e-wallet*, transfer bank, kartu kredit, cicilan, dan *refund*.

4. L4 - final method dan final class

Pada L4, program dikembangkan dengan menerapkan *keyword final* untuk membatasi pewarisan dan perubahan pada *class* maupun *method*. Selain itu, dibuat *class Address* yang bersifat *immutable*, yaitu nilai objeknya tidak dapat diubah setelah dibuat.

No.	Nama Class	Jenis	Fungsi
1	*ShippingService*	*Interface*	*Interface* untuk produk yang dapat dikirim.
2	*ShippingService*	*Class*	Class baru dengan sifat final dan atribut immutable.
3	*FreeShipping*	*Class*	Class sementara untuk menguji error ketika mencoba override method final.

a. ShippingService.java *(src/shop/shipping/ShippingService.java)*

Pada L4, method *calculateCost()* diubah menjadi *final*.

```
public final long calculateCost(int weightGrams) {
    int kg = (weightGrams + 999) / 1000;
    return kg * ratePerKg();
}
```

Penjelasan Kode;

Keyword *final* digunakan agar *method calculateCost()* tidak dapat di-*override* oleh *subclass*. Dengan demikian, semua jenis pengiriman tetap menggunakan rumus perhitungan ongkir yang sama, meskipun tarif setiap pengiriman berbeda.

b. FreeShipping *(src/shop/shipping/FreeShipping.java)*

Untuk menguji *method final*, dibuat class sementara

```
package shop.shipping;

public class FreeShipping extends ShippingService {

    @Override
    public long calculateCost(int weightGrams) {
        return 0;
    }}
```

Ketika program dikompilasi, muncul *error* karena *method calculateCost()* sudah menggunakan keyword *final*. Artinya, *subclass* tidak diperbolehkan mengubah implementasi *method* tersebut.

h. Address.java (*src\shop\model\ Address.java)*

Class Address menggunakan keyword *final* agar tidak dapat diwarisi oleh *class* lain. Setiap atribut menggunakan private final sehingga nilainya hanya dapat ditentukan melalui *constructor* dan tidak dapat diubah setelah objek dibuat.

Class ini hanya menyediakan *getter* tanpa *setter. Method toString()* digunakan untuk menampilkan alamat dalam format jalan, kota, dan kode pos.

```
package shop.model;
public final class Address {
    private final String street;
    private final String city;
    private final String postalCode;

    public Address(String street, String city, String postalCode) {
        if (street == null || street.isBlank()
                || city == null || city.isBlank()
                || postalCode == null || postalCode.isBlank()) {
            throw new IllegalArgumentException("Address cannot be empty"); }

        this.street = street;
        this.city = city;
        this.postalCode = postalCode; }

    public String getStreet() {
        return street;   }

    public String getCity() {
        return city;  }
```

```
public String getPostalCode() {
    return postalCode;    }
@Override
public String toString() {
    return street + ", " + city + " " + postalCode; }}
```

5. *L5 - Nested class: static nested, inner, dan anonymous class*

Pada L5, program dikembangkan dengan menerapkan tiga jenis *nested class*, yaitu *static nested class*, *inner class*, dan *anonymous class*. Ketiganya digunakan untuk mengelompokkan class sesuai fungsi dan hubungannya dengan class utama.

No.	Nama Class	Jenis	Fungsi
1	*Order*	*Class*	*Class* baru untuk menyimpan data pesanan.
2	*Order.Line*	*Class Order*	*Static nested class* untuk menyimpan detail barang pesanan.
3	*Order.Receipt*	*Class Order*	*Inner class* untuk membuat struk pesanan.
4	*ShopApp*	*Class*	Diubah untuk menggunakan *anonymous class* sebagai metode pembayaran.

i. *Order.java (src/shop/model/ShopApp.java)*

Class Order dibuat untuk menyimpan *ID* pesanan, alamat pengiriman, dan daftar barang yang dipesan.

```
package shop.model;
import java.util.;
...
```

```
public class Order {
    private final String orderId;
    private final Address shippingAddress;
    private final List<Line> lines = new ArrayList<>();

    public Order(String orderId, Address shippingAddress) {
        if (orderId == null || orderId.isBlank()
                || shippingAddress == null) {
            throw new IllegalArgumentException(
                "Order id and address are required" );   }

        this.orderId = orderId;
        this.shippingAddress = shippingAddress;  }

    public String getOrderId() {
        return orderId;   }
    public Address getShippingAddress() {
        return shippingAddress;   }

    public void addLine(Product product, int quantity) {
        lines.add(new Line(product, quantity));   }

    public List<Line> getLines() {
        return Collections.unmodifiableList(lines);    }
...
```

a. Static Nested Class : Line

Class Line dibuat di dalam *Order* dengan keyword *static*. Class ini menyimpan produk dan jumlah barang yang dipesan.

```
// static nested: tidak butuh objek Order
public static class Line {
    private final Product product;
    private final int quantity;

    public Line(Product product, int quantity) {
        if (product == null) {
            throw new IllegalArgumentException("Product must not be null");
        }
```

```
        if (quantity <= 0) {
            throw new IllegalArgumentException("Quantity must be positive");
        }
        this.product = product;
        this.quantity = quantity;    }

    public Product getProduct() { return product; }
    public int getQuantity() { return quantity; }

    public long subtotal() {
        return product.getFinalPrice() * quantity;  } }
```

b. *Method getItemsTotal()*

Method ini digunakan untuk menghitung total harga seluruh barang dalam pesanan.

```
public long getItemsTotal() {
    long total = 0;
    for (Line line : lines) {
        total += line.subtotal();
    }
    return total;
}
```

c. *Inner Class : Receipt*

Class Receipt dibuat tanpa *keyword static* karena membutuhkan akses ke data objek *Order* yang memilikinya**.**

```
 // inner class (tanpa static): membaca data Order lewat Order.this
    public class Receipt {
        public String toText() {
            StringBuilder sb = new StringBuilder();
            sb.append("STRUK PESANAN
").append(Order.this.orderId).append("\n");
            for (Line line : lines) {
                sb.append("- ").append(line.getProduct().getName())
                  .append(" x").append(line.getQuantity())
                  .append(" = Rp").append(line.subtotal()).append("\n");
            }
            sb.append("Total pesanan: Rp").append(getItemsTotal()).append("\n");
            sb.append("Kirim ke: ").append(Order.this.shippingAddress);
            return sb.toString();    } }}
```

Kemudian tambahkan *method* untuk membuat objek *Receipt*:

```
public Receipt createReceipt() {
    return new Receipt();
}
```

Inner class Receipt dapat mengakses atribut milik objek *Order* melalui *Order.this*, seperti *ID* pesanan dan alamat pengiriman.

j. Anonymous Class : ShopApp (shop/app/ShopApp.java)

Pada *ShopApp*, metode pembayaran dibuat menggunakan *anonymous class* yang mengimplementasikan *PaymentMethod*.

```
PaymentMethod voucher = new PaymentMethod() {
   @Override
   public String getName() {
      return "Voucher Promo Hari Ini";   }


   @Override
   public boolean pay(long amount) {
      return amount > 0; }};
```

b) Tugas

1. T1 - Evolusi API dan konflik diamond

Pada T1, program dikembangkan untuk menambahkan biaya administrasi pada setiap metode pembayaran.

a. T1.a – Menambahkan *Abstract Method* dan Mengamati *Error*

Pada tahap ini, *interface PaymentMethod* diubah dengan menambahkan *method abstract*.

```
public interface PaymentMethod {
   String getName();
   boolean pay(long amount);

   long adminFee(long amount);
}
```

Penambahan *method abstract* menyebabkan *class* yang mengimplementasikan *PaymentMethod* harus menyediakan implementasi *adminFee()*. Berdasarkan uji coba yang dilakukan *terdapat 4 eror* yang dialami, yaitu : *ShopApp.java, BankTransferPayment.java, CreditCardPayment.java,* dan *EwalletPayment.java.*

b. T1.b – Mengganti menjadi *Default method*

Untuk mengatasi error sebelumnya, *method adminFee()* diubah menjadi *default method*.

```
default long adminFee(long amount) {
   return 0;
}
```

Default method memiliki implementasi bawaan sehingga *class* yang mengimplementasikan *PaymentMethod* tidak wajib membuat implementasi sendiri, sehingga error sebelumnya dapat ditasi.

c. T1.c – *Override* Biaya Admin Sesuai Metode Pembayaran

Setiap metode pembayaran diberikan biaya administrasi yang berbeda.

i. CreditCardPayment.java

```
@Override
public long adminFee(long amount) {
   return amount * StudentInfo.CARD_ADMIN_PERCENT / 100;
}
```

Biaya admin kartu kredit dihitung berdasarkan persentase dari total pembayaran.

ii. BankTransferPayment.java

```
@Override
public long adminFee(long amount) {
   return StudentInfo.TRANSFER_ADMIN_FEE;
}
```

Biaya admin transfer bank menggunakan nominal tetap sesuai parameter NIM.

iii. EWalletPayment.java

Class EWalletPayment tidak perlu diubah karena menggunakan *default method* dari *PaymentMethod*, sehingga biaya adminnya adalah Rp0.

d. T1.d – Menambahkan *grandTotal()* pada *CheckoutService*

Pada tahap ini, *CheckoutService* diubah agar total pembayaran sudah mencakup biaya administrasi. Dengan menambahkan *method grandTotal()*.

```
public long grandTotal(
      List<Product> items,
      ShippingService shipping,
      PaymentMethod payment) {

   long amount = total(items, shipping);
   return amount + payment.adminFee(amount);
}
```

Kemudian mengubah *method checkout*() agar menggunakan *grandTotal()*

```
public boolean checkout(
      List<Product> items,
      ShippingService shipping,
      PaymentMethod payment) {

   if (items.isEmpty()) {
      return false;
   }

   return payment.pay(grandTotal(items, shipping, payment));
}
```

Method *grandTotal()* menghitung total belanja beserta ongkos kirim, kemudian menambahkan biaya admin sesuai metode pembayaran yang digunakan. Selanjutnya, *checkout()* menggunakan hasil perhitungan tersebut sebagai nominal pembayaran.

e. T1.e – Konflik Diamond pada *Default Method*

Pada tahap ini, ditambahkan *method policy*() pada dua *interface*, yaitu *RefundablePayment* dan *InstallmentPayment.*

```
// in RefundablePayment
default String policy() {
   return "Refund maksimal 7 hari";
}
// in InstallmentPayment
default String policy() {
   return "Cicilan maksimal " + maxInstallmentMonths() + " bulan";
}
```

Setelah kedua *method* ditambahkan. Akan muncul error pada *CreditCardPayment* karena *class* tersebut mengimplementasikan kedua *interface* yang memiliki *default method* dengan nama dan signature sama.

2. T2 - Merancang sistem notifikasi pesanan

Pada T2, program dikembangkan dengan membuat sistem notifikasi untuk mengirim informasi perubahan status pesanan kepada pelanggan melalui beberapa kanal, yaitu Email, SMS, dan WhatsApp.

No.	Nama Class	Jenis	Fungsi
1	*CustomerContact*	*Class*	Menyimpan data nama, *email,* dan nomor telepon pelanggan.
2	*NotificationChannel*	*Interface*	*Interface* sebagai kontrak untuk setiap kanal notifikasi.
3	*AbstractChannel*	*Abstract Class*	*Abstract class* yang menyediakan proses pengiriman notifikasi secara umum
4	*EmailChannel*	*Class*	Mengirim notifikasi melalui email.
5	*SmsChannel*	*Class*	Mengirim notifikasi melalui SMS dengan batas panjang pesan.
6	*WhatsAppChannel*	*Class*	Mengirim notifikasi melalui WhatsApp.
7	*NotificationService*	*Class*	Mengelola kanal dan mengirim notifikasi ke pelanggan.

Berikut merupakan rancangan *uml* yang menggambarkan hubungan antar *class* pada sistem notifikasi.



Berikut merupakan implementasi kode program berdasarkan rancangan yang telah dibuat:

a. *CustomerContact*.java

Class ini menggunakan *final* agar tidak dapat diwarisi. Atribut dibuat *private final* dan hanya menyediakan *getter* sehingga data kontak tidak dapat diubah setelah objek dibuat.

```
package shop.notification;
public final class CustomerContact {
    private final String name;
    private final String email;
    private final String phone;
```

```
public CustomerContact(String name, String email, String phone) {
    if (name == null || name.isBlank()) {
        throw new IllegalArgumentException("Name is required");
    }
    this.name = name;
    this.email = email == null ? "" : email;
    this.phone = phone == null ? "" : phone;    }

public String getName() { return name; }
public String getEmail() { return email; }
public String getPhone() { return phone; }}
```

b. *NotificationChannel*.java

Interface ini menjadi kontrak bagi seluruh kanal notifikasi. Dimana setiap kanal wajib memiliki nama kanal dan *method send*() untuk mengirim pesan kepada pelanggan.

```
package shop.notification;
public interface NotificationChannel {
    String channelName();
    boolean send(CustomerContact contact, String message);}
```

c. *AbstractChannel*.java

Abstract class menyediakan proses pengiriman notifikasi yang digunakan bersama oleh seluruh kanal.

```
package shop.notification;
public abstract class AbstractChannel
        implements NotificationChannel {

    private int sentCount;
    private String lastDelivered;
    public final boolean send(
            CustomerContact contact, String message) {
        if (contact == null || message == null
                || message.isBlank()) {
            return false;   }
```

```
        String recipient = resolveRecipient(contact);
        if (recipient == null || recipient.isBlank()) {
            return false;   }

        String prepared = prepare(message);

        if (!deliver(recipient, prepared)) {
            return false; }
        sentCount++;
        lastDelivered = prepared;
        return true;  }

    protected abstract String resolveRecipient(
            CustomerContact contact);
    protected String prepare(String message) {
        return message;    }

    protected abstract boolean deliver(
            String recipient, String message);
    public int getSentCount() {
        return sentCount;   }
    public String getLastDelivered() {
        return lastDelivered;   }}
```

d. *EmailChannel*.java

Email hanya dapat dikirim jika alamat email pelanggan mengandung karakter @.

```
package shop.notification;
public class EmailChannel extends AbstractChannel {
    @Override
    public String channelName() {
        return "EMAIL";}
    @Override
    protected String resolveRecipient(CustomerContact contact) {
        String email = contact.getEmail();
        return email.contains("@") ? email : null;   }
```

```
@Override
protected boolean deliver(String recipient, String message) {
    System.out.println("[EMAIL to " + recipient + "] " + message);
    return true;   }}
```

e. SmsChannel.java

SMS hanya dikirim ke nomor yang diawali *08*. Jika pesan melebihi batas karakter, pesan akan dipotong dan diakhiri dengan ... sehingga panjangnya tetap sesuai batas yang ditentukan.

```
package shop.notification;
public class SmsChannel extends AbstractChannel {


    private final int maxLength;
    public SmsChannel(int maxLength) {
        this.maxLength = maxLength;}

    @Override
    public String channelName() {
        return "SMS";
    @Override
    protected String resolveRecipient(CustomerContact contact) {
        String phone = contact.getPhone();
        return phone.startsWith("08") ? phone : null;    }

    @Override
    protected String prepare(String message) {
        if (message.length() > maxLength) {
            return message.substring(0, maxLength - 3) + "...";  }
        return message; }
    @Override
    protected boolean deliver(String recipient, String message) {
        System.out.println("[SMS to " + recipient + "] " + message);
        return true;    }}
```

f. *WhatsAppChannel*.java

WhatsApp menerima nomor yang diawali 08 atau +62 dengan panjang minimal 10 karakter. Pesan juga diberi awalan [*Toko UMKM]*.

```
package shop.notification;
public class WhatsAppChannel extends AbstractChannel {
   @Override
   public String channelName() {
      return "WA";}

   @Override
   protected String resolveRecipient(CustomerContact contact) {
     String phone = contact.getPhone();
      if (phone.length() < 10) {
         return null;  }
      if (phone.startsWith("08") || phone.startsWith("+62")) {
         return phone; }

      return null;    }

   @Override
   protected String prepare(String message) {
      return "[Toko UMKM] " + message;    }

   @Override
   protected boolean deliver(String recipient, String message) {
      System.out.println("[WA to " + recipient + "] " + message);
      return true;   }}
```

g. NotificationService.java

NotificationService menyimpan daftar kanal notifikasi dan mengirim pesan melalui seluruh kanal yang sudah didaftarkan. *Method broadcast()* mengembalikan jumlah kanal yang berhasil mengirim pesan.

```
package shop.notification;
import java.util.ArrayList;
import java.util.List;

public class NotificationService {
   private final List<NotificationChannel> channels =
         new ArrayList<>();
   public void register(NotificationChannel channel) {
      if (channel != null) {
         channels.add(channel);    }    }

   public int broadcast(CustomerContact contact, String message) {
      int success = 0;

      for (NotificationChannel channel : channels) {
         if (channel.send(contact, message)) {
            success++;  }   }

      return success;} }
```

h. TugasApp.java (src/shop/app/TugasApp.java)

Pada *TugasApp*, ketiga kanal notifikasi didaftarkan bersama satu kanal tambahan berupa *anonymous class* untuk mencatat pesan ke konsol. Kemudian, sistem mengirim pesan panjang kepada dua pelanggan dengan data kontak yang berbeda. Tujuannya untuk menguji apakah setiap kanal dapat menentukan penerima sesuai ketentuannya.

```
package shop.app;

import java.util.List;
...
public class TugasApp {
   public static void main(String[] args) {
         System.out.println("Tugas  oleh  " + StudentInfo.NAME + " (" +
StudentInfo.NIM + ")");
```

```
        Product coffee = new PhysicalProduct("KOPI-001", "Kopi Arabika Gayo
250g", 85_000, 300);
        Product rendang = new PhysicalProduct("RDG-002", "Rendang Kemasan
250g", 120_000, 400);
         Product ebook = new DigitalProduct("EBK-003", "E-book Resep Kue
Nusantara", 50_000,
                "https://toko-umkm.example/unduh/ebk-003");
        List<Product> cart = List.of(coffee, rendang, ebook);

        CheckoutService checkout = new CheckoutService();
        ShippingService express = new ExpressShipping();

        // 1. Grand total
        EWalletPayment wallet = new EWalletPayment("DompetKu", 300_000);
        PaymentMethod transfer = new BankTransferPayment("Bank Nagari");
           CreditCardPayment card = new CreditCardPayment("Siti Rahma",
1_000_000);

        System.out.println("\n== T1: Total dengan biaya admin (Ekspres) ==");
        printTotal(checkout, cart, express, wallet);
        printTotal(checkout, cart, express, transfer);
        printTotal(checkout, cart, express, card);

        // 2. policy() kartu kredit dan e-wallet
        System.out.println("Kebijakan kartu kredit : " + card.policy());
        System.out.println("Kebijakan e-wallet : " + wallet.policy());

        // 3. Daftar tiga kanal + satu kanal log (anonymous class)
        NotificationService service = new NotificationService();
        service.register(new EmailChannel());
        service.register(new SmsChannel(StudentInfo.SMS_MAX_LENGTH));
        service.register(new WhatsAppChannel());
        service.register(new NotificationChannel() {

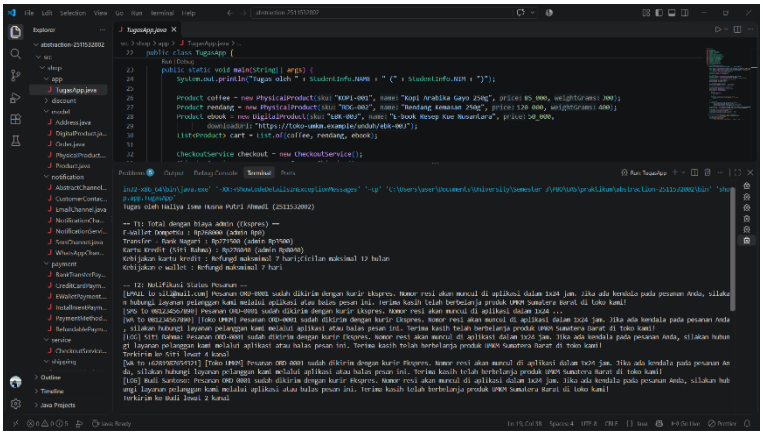
           @Override
           public String channelName() {
              return "Log";  }
```

```
        @Override
        public boolean send(CustomerContact contact, String message) {
            System.out.println("[LOG] " + contact.getName() + ": " + message);
            return true;   }     });

        // 4. Pesan panjang
        String message = "Pesanan ORD-0001 sudah dikirim dengan kurir Ekspres. "
                + "Nomor resi akan muncul di aplikasi dalam 1x24 jam. "
                 + "Jika ada kendala pada pesanan Anda, silakan hubungi layanan pelanggan "
                + "kami melalui aplikasi atau balas pesan ini. "
                 + "Terima kasih telah berbelanja produk UMKM Sumatera Barat di toko kami!";
                 CustomerContact siti = new CustomerContact("Siti Rahma", "siti@mail.com", "081234567890");
        CustomerContact budi = new CustomerContact("Budi Santoso", "", "+6281987654321");

        System.out.println("\n== T2: Notifikasi Status Pesanan ==");
        int sitiCount = service.broadcast(siti, message);
        System.out.println("Terkirim ke Siti lewat " + sitiCount + " kanal");
        int budiCount = service.broadcast(budi, message);
       System.out.println("Terkirim ke Budi lewat " + budiCount + " kanal"); }

    private static void printTotal(CheckoutService checkout, List<Product> cart,
        ShippingService shipping, PaymentMethod payment) {
        long total = checkout.total(cart, shipping);
        long admin = payment.adminFee(total);
        System.out.println(payment.getName() + " : Rp"
                + checkout.grandTotal(cart, shipping, payment) + " (admin Rp" + admin + ")");}}
```



3. *T3 - Code review BedShop*

Program dapat dikompilasi dan dijalankan, tetapi memiliki beberapa masalah desain yang menyebabkan *bug*. Seteah itu, bug atau error di identifikasi dan diperbaiki melalui *src/review/fixed/BadShopRefactored.java,* dan tidak mengubah *BadShop.java* asli.

```
package review.fixed;

import java.util.ArrayList;

import java.util.List;

// Masalah 1: interface dipecah per kemampuan

interface Payable {

    boolean pay(long amount);

    String printInvoice();}

interface Refundable extends Payable {

    boolean refund(long amount);}


// Voucher membayar, tidak ada ship/refund yang melempar exception

class GiftVoucher implements Payable {

    private long balance = 100_000; // 7.Tipe Voucher double, ganti ke long
```

```
    @Override
    public boolean pay(long amount) {
        if (amount <= 0 || amount > balance) {
            return false;       }
        balance -= amount;
        return true;  }
    @Override
    public String printInvoice() {
        return "sisa Rp" + balance;  }}
// Masalah 2 dan 3: rumus dikunci, tarif diisi tiap kurir
abstract class Courier {
    private final String name;
    protected Courier(String name) {
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Name must not be blank");    }
        this.name = name;    }

    public String getName() {
        return name;    }

    protected abstract long ratePerKg();
    public final long cost(int kilograms) {
        if (kilograms <= 0) {
            return 0;       }
        return kilograms * ratePerKg();    }}
```

```
class RegularCourier extends Courier {
    RegularCourier() { super("Reguler"); }
    @Override
    protected long ratePerKg() { return 10_000; }}

class ExpressCourier extends Courier {
    ExpressCourier() { super("Ekspres"); }
    @Override
    protected long ratePerKg() { return 18_000; }}

class SameDayCourier extends Courier {
    SameDayCourier() { super("Same Day"); }
    @Override
    protected long ratePerKg() { return 30_000; }}

// Masalah 6: static nested, tidak butuh objek Cart
class Cart {
    private final List<Item> items = new ArrayList<>();
    static class Item {
        private final String name;
        private final long price;
        private final int quantity;
        Item(String name, long price, int quantity) {
            if (name == null || name.isBlank() || price < 0 || quantity <= 0) {
                throw new IllegalArgumentException("Invalid item");     }
            this.name = name;  this.price = price;    this.quantity = quantity;    }
```

```
    long subtotal() {
        return price * quantity;}
    String getName() {
        return name;       }   }
  void add(Item item) {
    items.add(item);   }
  long total() {
    long total = 0;
    for (Item item : items) {
        total += item.subtotal();   }
    return total;   }}

// Masalah 4: immutable
final class Address {
  private final String street;
  private final String city;
  Address(String street, String city) {
    if (street == null || street.isBlank() || city == null || city.isBlank()) {
        throw new IllegalArgumentException("Street and city must not be blank");       }
    this.street = street;
    this.city = city;   }
  String getStreet() {
    return street;   }
  String getCity() {
    return city;   }
```

```
    @Override
    public String toString() {
        return street + ", " + city;    }}
class SimpleOrder {
    private final String id;
    private final Address address;
    SimpleOrder(String id, Address address) {
        this.id = id;
        this.address = address;    }
    @Override
    public String toString() {
        return id + " -> " + address;    }}

// Masalah 5: persentase abstract, perhitungan default
interface Discount {
    int percent();
    default long apply(long price) {
        int safe = Math.max(0, Math.min(percent(), 100));
        return price - price * safe / 100;  }}
class MemberDiscount implements Discount {
    @Override
    public int percent() { return 5; }}
class FlashSaleDiscount implements Discount {
    @Override
    public int percent() { return 30; }}
public class BadShopRefactored {
```

```
public static void main(String[] args) {
    Cart cart = new Cart();
    cart.add(new Cart.Item("Kopi Gayo", 85_000, 2));
    System.out.println("Total cart: " + cart.total());

    Courier courier = new ExpressCourier();
    System.out.println("Ongkir " + courier.getName() + " 2 kg: " + courier.cost(2));

    Address home = new Address("Jl. Sudirman 10", "Padang");
    SimpleOrder first = new SimpleOrder("ORD-1", home);
    SimpleOrder second = new SimpleOrder("ORD-2",
        new Address(home.getStreet(), "Bukittinggi")); // objek baru, home tidak berubah
    System.out.println(first);        System.out.println(second);
    Discount flashSale = new FlashSaleDiscount();
    System.out.println("Harga flash sale: " + flashSale.apply(100_000));
    Payable voucher = new GiftVoucher();
    System.out.println(voucher.pay(30_000) + " | " + voucher.printInvoice());
    // voucher.ship(100); // compile error: Payable tidak punya ship()    }}
```

Berikut merupakan hasil yang telah diperbaiki, ongkir Ekspres menjadi Rp36.000, alamat ORD-1 tetap Padang, dan pembayaran voucher tidak menyebabkan *exception*.

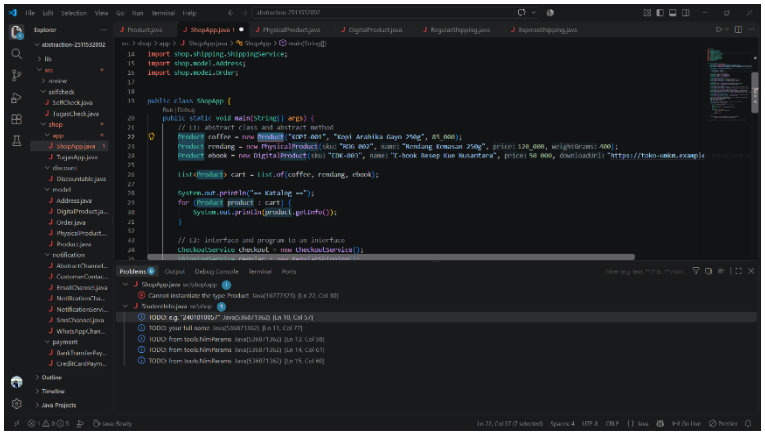
```
'C:\Users\user\Documents\University\Semester 3\PBO\UAS\praktikum\abstraction-2511532002\bin'
Total cart: 170000
Ongkir Ekspres 2 kg: 36000
ORD-1 -> Jl. Sudirman 10, Padang
ORD-2 -> Jl. Sudirman 10, Bukittinggi
Harga flash sale: 70000
true | sisa Rp70000
PS C:\Users\user\Documents\University\Semester 3\PBO\UAS\praktikum\abstraction-2511532002>
```

C. TUGAS DAN LATIHAN

1. Error Compiler

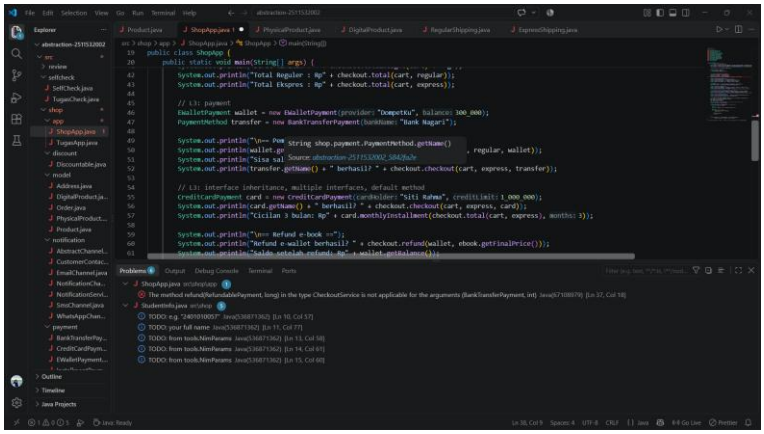
1) L1

Pada latihan L1 terdapat *error compiler* yang berkaitan dengan penerapan *abstract class* dan *abstract method*. Salah satu *error* terjadi ketika mencoba membuat objek secara langsung dari *class Product* menggunakan *new Product*(...). *Compiler* menolak perintah tersebut karena *Product* merupakan *abstract class* yang tidak dapat diinstansiasi secara langsung. Objek hanya dapat dibuat melalui *subclass* konkret seperti *PhysicalProduct* dan *DigitalProduct*.



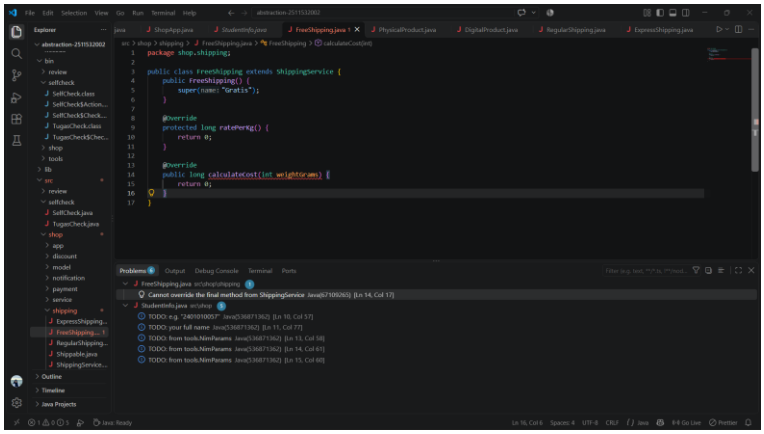
2) L3

Pada latihan L3 terdapat *error compiler* yang berkaitan dengan penerapan *interface inheritance*. *Error* terjadi ketika mencoba memanggil *checkout.refund(new BankTransferPayment("Bank Nagari"), 1_000). Compiler* menolak perintah tersebut karena *method refund*() pada *CheckoutService* hanya menerima parameter bertipe *RefundablePayment,* sedangkan *BankTransferPayment* hanya mengimplementasikan *PaymentMethod*. Dengan demikian, kesalahan terdeteksi saat penulisan kode, sebelum program dijalankan, tanpa memerlukan satu pun *if*.



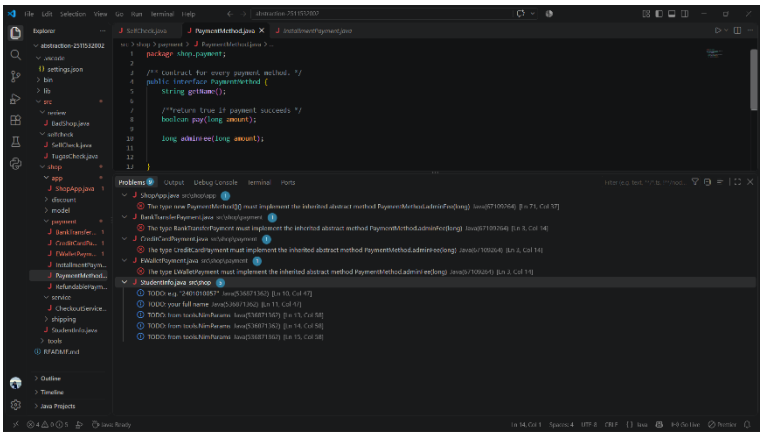
3) L4

Pada latihan L4 terdapat *error compiler* yang berkaitan dengan penerapan *final method*. Dimana dibuat untuk sementara *class FreeShipping extends ShippingService* yang meng-*override calculateCost*. *Error* terjadi ketika *class FreeShipping* mencoba menimpa (*override*) *calculateCost()* milik *ShippingService*. *Compiler* menolaknya karena *method* tersebut bertanda *final*, sehingga rumus ongkos kirim tidak dapat diganti oleh subkelas. Subkelas tetap bebas mengatur tarif lewat *ratePerKg()*.



4) T1.a

Pada tugas T1.a, penambahan *abstract method adminFee*(*long amount*) ke *PaymentMethod* menimbulkan *error compiler* pada setiap *class* yang mengimplementasikan antarmuka tersebut, karena semuanya wajib mengisi *method* baru itu. *Anonymous class voucher* di *ShopApp* juga ikut *error* karena *anonymous class* tetap merupakan *class* konkret yang mengimplementasikan *PaymentMethod*, hanya saja tidak memiliki nama..



Terdapat 4 error yang terjadi, yaitu diantaranya:

1. *ShopApp.java (src\shop\app)*
   The type new PaymentMethod(){} must implement the inherited abstract method PaymentMethod.adminFee(long)

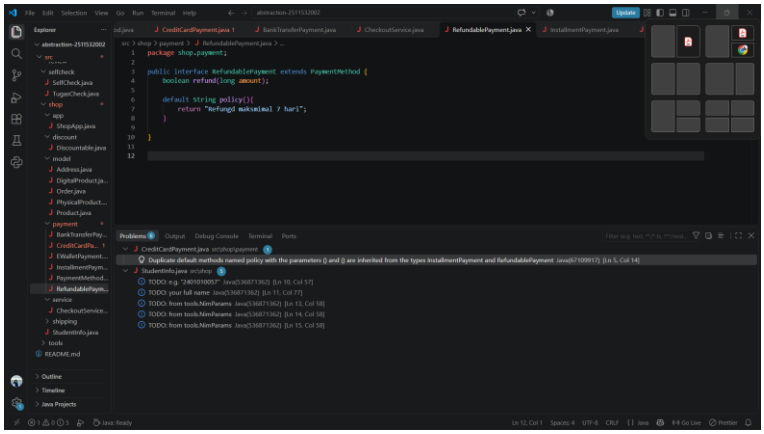
2. *BankTransferPayment.java (src\shop\payment)*
   The type new PaymentMethod(){} must implement the inherited abstract method PaymentMethod.adminFee(long)

3. *CreditCardPayment.java (src\shop\payment)*
   The type CreditCardPayment must implement the inherited abstract method PaymentMethod.adminFee(long)

4. *EwalletPayment.java (src\shop\payment)*
   The type EWalletPayment must implement the inherited abstract method PaymentMethod.adminFee(long)

5) T1.e

Pada tugas *T1.e* terdapat *error compiler* yang berkaitan dengan konflik diamond. *Error* terjadi pada *CreditCardPayment* karena *class* tersebut mewarisi dua *default method policy()* dari dua antarmuka berbeda, yaitu *RefundablePayment* dan *InstallmentPayment*. *Compiler* tidak memilih salah satu secara otomatis, sehingga programmer harus menyelesaikannya dengan menimpa *policy*() di *CreditCardPayment*.



Keterangan:

CreditCardPayment.java (src\shop\payment)Duplicate default methods named policy with the parameters () and () are inherited from the types InstallmentPayment and RefundablePayment

2. Jawaban Pertanyaan

1) T1 - Evolusi API dan konflik diamond

Jawab di laporan:

1) Ada berapa *error*, *class* mana saja, dan mengapa *anonymous class voucher* di *ShopApp* juga ikut rusak?

Jawab:

Terdapat 4 *error*, masing-masing pada *class* konkret yang mengimplementasikan *PaymentMethod*, yaitu *EWalletPayment*, *BankTransferPayment*, *CreditCardPayment*, dan *anonymous class voucher* di *ShopApp*. Pesan error menyatakan bahwa *class* tersebut harus mengimplementasikan *method abstrak adminFee(long)* yang

diwarisi dari *PaymentMethod*. Sedangkn untuk *Anonymous class* tetap merupakan *class* biasa, hanya saja tidak diberi nama. Bentuk *new PaymentMethod() { ... }* berarti membuat *class* baru yang mengimplementasikan *PaymentMethod*. Aturan Java berlaku sama: setiap *class* wajib mengisi semua *method* abstrak dari antarmuka yang ditandatanganinya. *Voucher* hanya mengisi *getName()* dan *pay(),* sehingga *adminFee*() belum ada dan penyusun program (*compiler*) menolaknya

Menambah *method* abstrak ke antarmuka yang sudah dipakai banyak *class* disebut *breaking change*, karena semua pelaksananya harus diubah. Masalah ini diselesaikan pada T1.b dengan menjadikannya *default method*.

2) Mengapa *EWalletPayment* tidak perlu *meng-override policy*()?

Jawab:

EWalletPayment hanya mengimplementasikan *RefundablePayment*, sehingga hanya mewarisi satu *versi policy*(). Tidak ada pilihan lain, maka tidak terjadi kebingungan dan *compiler* langsung memakai versi itu, yaitu "Refund maksimal 7 hari". Konflik hanya muncul bila sebuah *class* mewarisi *default method* yang bernama dan berparameter sama dari dua antarmuka atau lebih. *CreditCardPayment* mengalaminya karena mengimplementasikan *RefundablePayment* dan *InstallmentPayment* sekaligus.

2) Mengapa Java tidak memilih salah satu default secara otomatis?

Jawab:

Karena memilih otomatis berarti menebak, dan tebakan yang keliru membuat program berjalan dengan perilaku yang tidak dimaksudkan tanpa ada peringatan. Kedua antarmuka kedudukannya setara, tidak ada

yang lebih khusus, sehingga tidak ada aturan yang adil untuk mengutamakan salah satunya. Memilih berdasarkan urutan penulisan implements juga berbahaya, sebab menukar urutan saja dapat mengubah hasil program. *Java* memilih jalan aman: memaksa programmer menentukan sendiri (memilih salah satu, menggabungkan, atau menulis perilaku baru), sehingga masalahnya ketahuan saat penyusunan program, bukan saat program berjalan. Pada Cr*editCardPaymen*t, penyelesaiannya adalah menggabungkan keduanya dengan *RefundablePayment.super.policy()* dan *InstallmentPayment.super.policy().*

3) T2 - Merancang sistem notifikasi pesanan

Jawab di laporan:

1. Mengapa ada *interface* dan *abstract class*, bukan salah satunya saja?

   Jawab:

   Keduanya memiliki tugas yang berbeda. Antarmuka *NotificationChannel* menetapkan kontrak "apa yang dapat dilakukan sebuah kanal", yaitu *channelName*() dan *send*(...), sehingga siapa pun dapat menjadi kanal, termasuk *anonymous class* kanal *log* di *TugasApp* yang tidak mewarisi apa pun. Kelas *abstrak AbstractChannel* menampung langkah kerja yang sama untuk semua kanal: menolak pesan kosong, mencari alamat tujuan, menyiapkan pesan, mengirim, lalu mencatat jumlah dan pesan terakhir. Jika hanya ada antarmuka, kode yang sama harus ditulis ulang pada tiap kanal. Jika hanya ada kelas abstrak, kanal yang sudah mewarisi kelas lain atau berbentuk *anonymous class* tidak dapat ikut serta.

2. Apa akibatnya jika *send(...)* tidak dibuat *final*?

   Jawab:

   Subkelas dapat menimpa *send(...) dan* mengubah urutan langkahnya. Akibatnya, sebuah kanal baru bisa melewati pemeriksaan pesan kosong, lupa menambah *sentCount*, atau tidak mencatat *lastDelivered*. Perilaku antar kanal menjadi tidak seragam dan jaminan dari kelas induk hilang. Dengan final, urutan kerja terkunci dan subkelas hanya boleh mengisi bagian yang berbeda lewat *resolveRecipient*(...), *deliver*(...), dan *prepare*(...). Pola ini disebut *Template Method*.

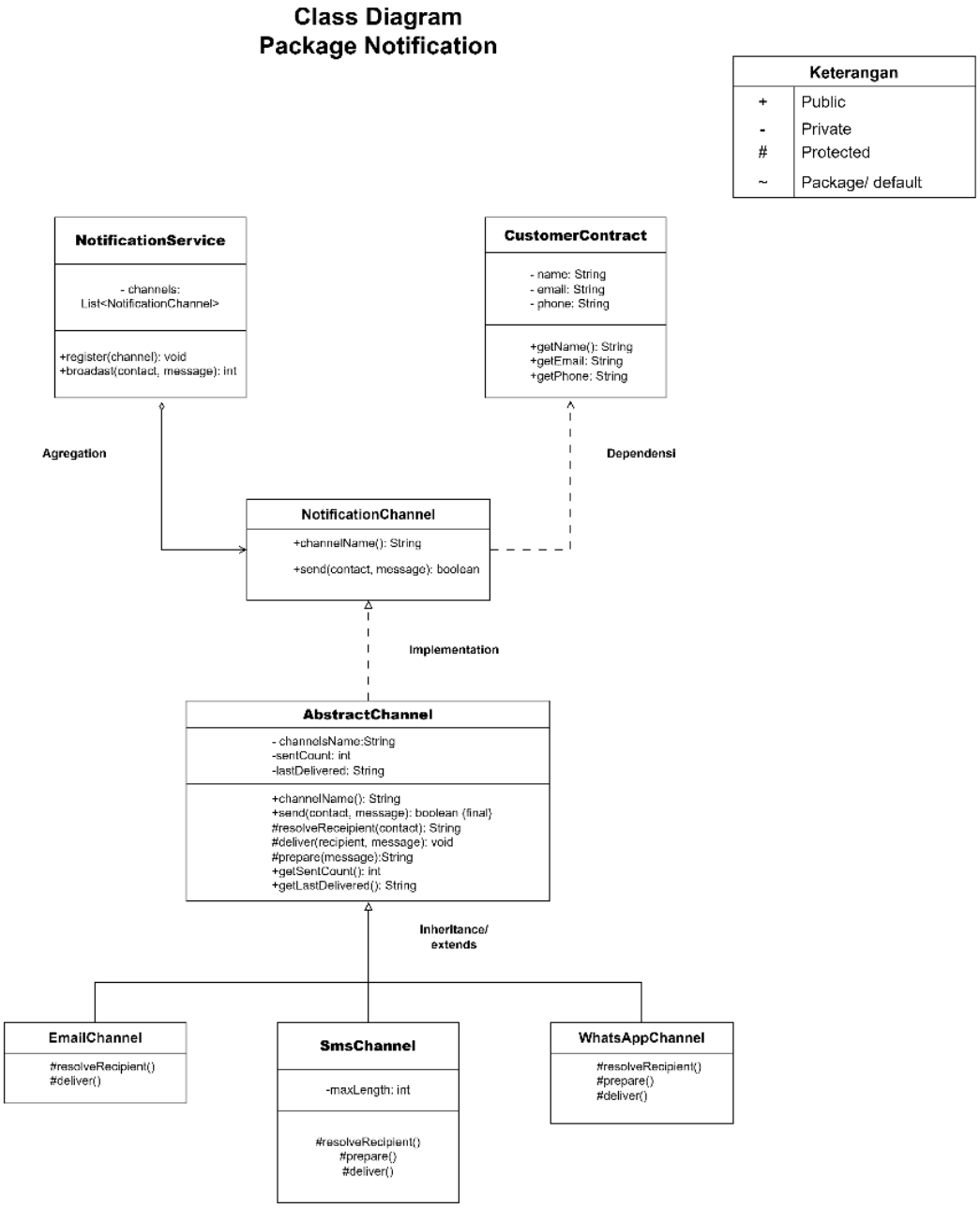
3. Mengapa *NotificationService* menyimpan *List<NotificationChannel>,* bukan *List<AbstractChannel>?*

   Jawab:

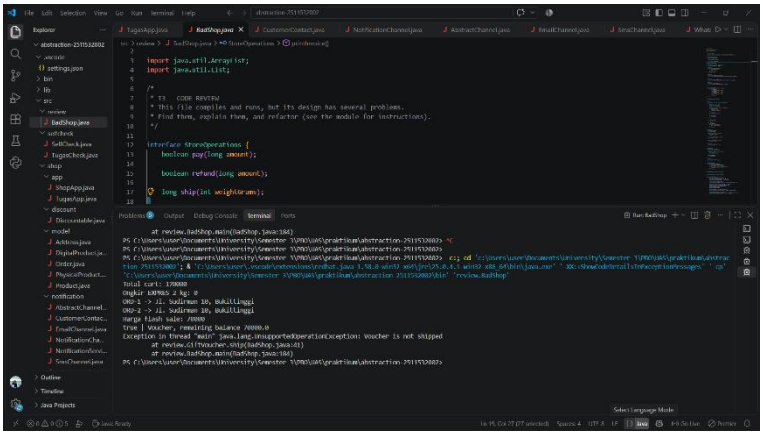
   Dengan tipe antarmuka, *NotificationService* menerima kanal apa pun yang memenuhi kontrak, termasuk kanal *log* berupa *anonymous class* dan kanal masa depan seperti *Telegram* yang tidak mewarisi *AbstractChannel*. Jika tipenya *AbstractChannel*, semua kanal tersebut ditolak penyusun program, dan layanan menjadi terikat pada satu hierarki kelas. Inilah makna *program to an interface*: pengirim hanya mengenal abstraksi, sehingga kanal baru cukup ditambahkan sebagai kelas baru tanpa mengubah *NotificationService*.

4) Diagram *UML* T2

Berikut merupakan diagram *UML* yang menggambarkan rancangan sistem notifikasi pesanan.



5) T3 - Code review BadShop



Saat program *BadShop.java* dijalankan, erlihat tiga gejala utama: ongkir ekspres tidak dihitung, alamat kedua pesanan berubah bersamaan, dan program berhenti karena *exception* ketika voucher diproses sebagai barang kiriman.

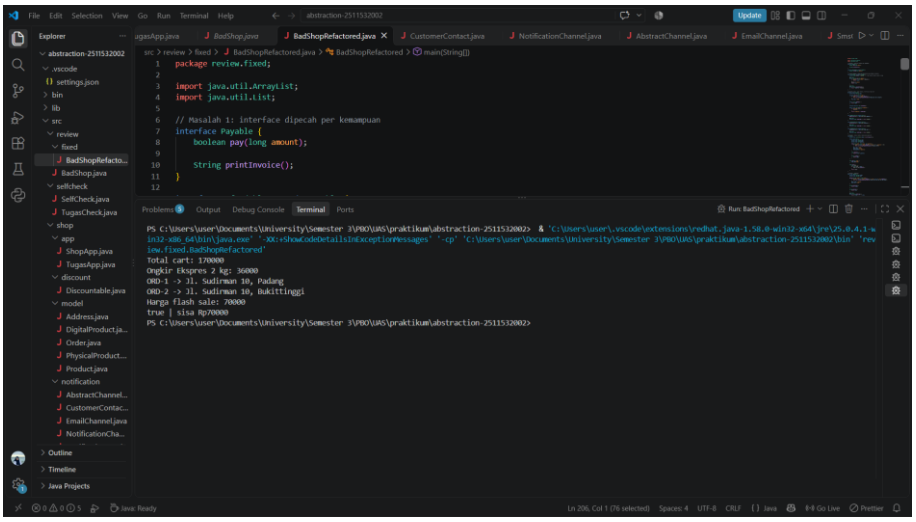
Berikut merupakan tabel masalah yang telah ditemukan, masalah-masalah tersebut berkaitan dengan materi *abstraction*, *interface*, *abstract class, default method, final*, dan *nested class.*

No	Lokasi (*Class/method*)	Masalah	Dampak nyata (*bug* atau resiko)	Perbaikan yang diusulkan
1.	*Interface StoreOperations, class GiftVoucher*	*Interface* terlalu gemuk. *Voucher* dipaksa punya *refund* dan *ship*, lalu keduanya melempar *UnsupportedOperationException*.	Program *crash* di *voucher.ship(100)*. *Compiler* tidak bisa mencegahnya karena tipe *StoreOperations* menjanjikan semua kemampuan.	Pecah menjadi *interface* kecil (*Payable, Refundable*). Voucher hanya *implements Payable*, sehingga *ship* ditolak *compiler*.
2	*Courier.cost (String type, int kilograms)*	Jenis kurir dipilih dengan *switch* pada *String* bebas, dan *default* mengembalikan 0.	Output Ongkir EXPRES 2 kg: 0, jadi toko rugi tanpa error. Kurir baru juga memaksa mengubah *switch*.	Satu *class* per kurir dengan *ratePerKg*() sendiri. Jenis kurir ditentukan oleh tipe, bukan *String*.
3	*abstract class Courier dan JneCourier*	*Abstract class* tanpa *abstract method,* dan *JneCourier* kosong. Rumus ongkir *public* dan tidak *final*, sehingga *subclass* bebas mengganti rumusnya.	*Abstract class* tidak memaksa *subclass* memutuskan apa pun, dan rumus bersama tidak terlindungi.	dijadikan *ratePerKg() protected abstract, cost(...)* dijadikan *final.*

4	*class Address dan SimpleOrder.getAddress()*	Address *mutable*: *field* tidak *final*, ada *setter*, *class* tidak final. Satu objek *home* dipakai bersama dua pesanan.	Mengubah kota *ORD-2* ikut mengubah *ORD-1*: keduanya tercetak Bukittinggi, pengiriman bisa nyasar.	Mengubah kota *ORD-2* ikut mengubah *ORD*-1: keduanya tercetak Bukittinggi, pengiriman bisa nyasar.
5	*interface Discount.apply(...)*	*Default method* memakai *this instanceof MemberDiscount* dan *FlashSaleDiscount*, jadi *interface* mengenal implementasinya. Angka 5 dan 30 tertanam sebagai magic number.	Diskon baru tidak jalan. *Interface* jadi bergantung pada *class* turunannya.	*int percent()* dijadikan *method abstract*, dan *apply(...) default* menggunakannya.
6	*Cart.Item*	*Inner class (non-static)* padahal tidak membaca data *Cart*.	Menyimpan referensi tersembunyi ke *Cart* dan memaksa sintaks c*art.new Item(...),* boros memori dan membingungkan.	Jadikan *static class Item.*
7	*GiftVoucher.balance bertipe double*	Uang memakai *double,* padahal aturan modul uang bertipe *long.*	*Output remaining balance 70000.0*, dan berisiko kesalahan pembulatan pecahan.	Ganti ke *long*

Setelah dilakukan perbaikan dan *refactoring* pada kode program, dapat dilihat pada *output* berikut bahwa program telah berhasil dijalankan sesuai dengan hasil yang diharapkan. Permasalahan dan *error* yang ditemukan pada tahap *code review* sebelumnya telah berhasil diatasi melalui penerapan konsep *abstraction, interface*, dan prinsip pemrograman berorientasi objek. Dengan demikian, seluruh perbaikan yang dilakukan telah menjawab permasalahan yang ditemukan dan program dapat berjalan

tanpa mengalami error dan hasil sesuai logika yang dirapkan.



6) T4 - Refleksi: kapan memakai apa

Berikut isi dari T4 – Refleksi: Kapan Memakai Apa berdasarkan

a) Abstract class vs Interface

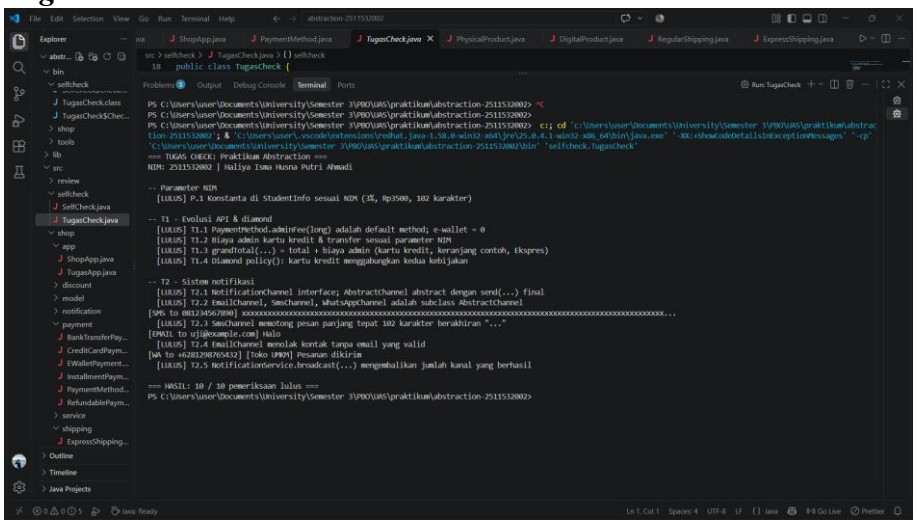
Pertanyaan desain	***Abstract class* atau *interface***	**Alasan**	**Contoh**
Subclass berbagi *field* dan konstruktor	*Abstract class*	Dapat menyimpan *field* dan konstruktor yang diwarisi oleh *subclass*.	*Product* memiliki *field sku*, *name*, dan *price* yang digunakan oleh *PhysicalProduct* dan *DigitalProduct*.
Satu *class* perlu beberapa kemampuan sekaligus	*Interface*	Satu *class* dapat mengimplementasikan beberapa *interface* sekaligus.	*CreditCardPayment* mengimplementasikan *RefundablePayment* dan *InstallmentPayment*.
Rumus bersama yang tidak boleh diubah subclass	*Abstract class*	Dapat menggunakan *final method* agar *subclass* tidak dapat mengubah implementasi rumus.	*ShippingService* memiliki *method calculateCost()* yang bersifat *final*.
Kontrak yang perlu ditambah *method* baru di	*Abstract class*	Dapat menggunakan *default method* sehingga penambahan *method* tidak mewajibkan seluruh *class* implementasi mengubah kodenya.	*PaymentMethod* memiliki *default method adminFee()*.

b) Jenis Nested class

Kondisi	***Static nested, inner, atau anonymous***	**Alasan**	**Contoh**
Class pembantu tidak membaca data objek luar	*Static nested*	Tidak membutuhkan objek dari *class* luar untuk dibuat atau digunakan.	*Order.Line* menyimpan informasi produk dan jumlah barang.
Objek harus membaca field objek luarnya	*Inner*	Dapat mengakses *field* milik objek luar melalui *Order.this*.	*Order.Line* menyimpan informasi produk dan jumlah barang.
Implementasi sekali pakai, tidak perlu nama	*Anonymous*	Dapat langsung membuat implementasi *interface* tanpa membuat *class* terpisah.	*Anonymous class PaymentMethod* untuk *voucher* di *ShopApp*.

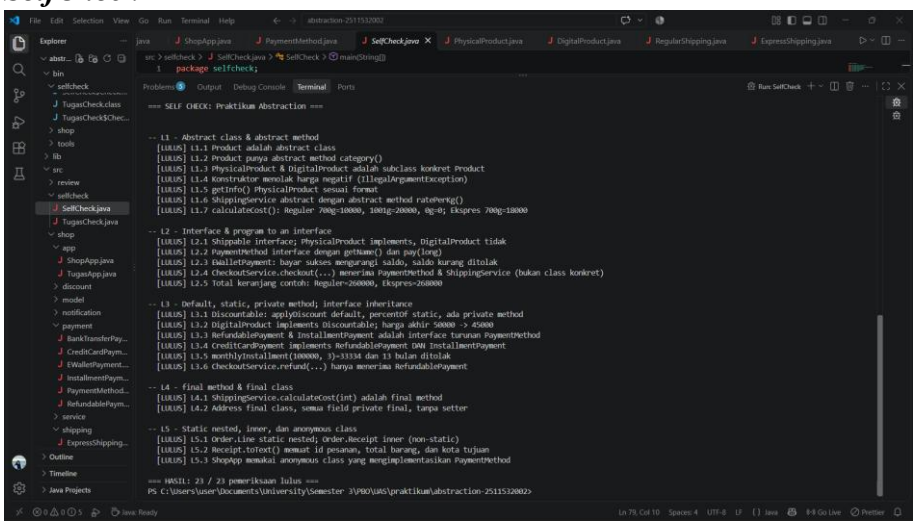
3. Tangkapan Layar

1) Tugas Check



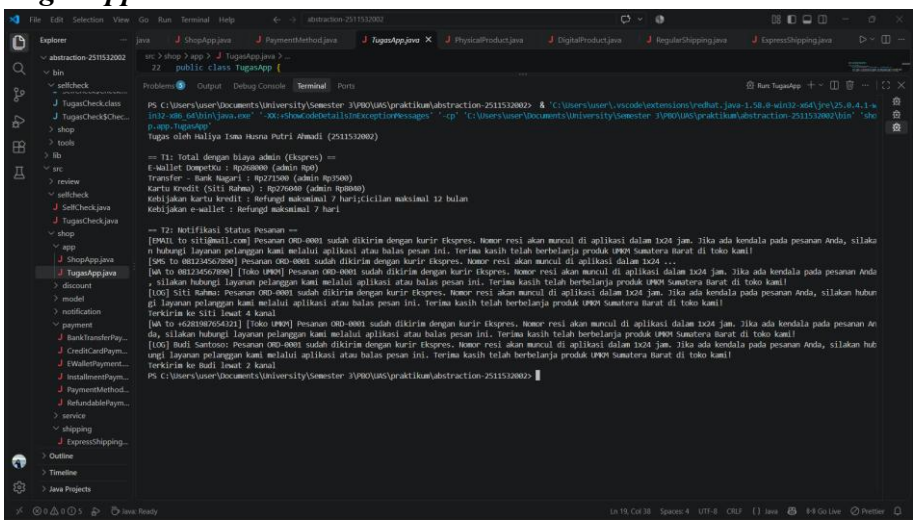
Berikut merupakan tangkapan layar hasil pengujian *Tugas Check* yang menunjukkan bahwa seluruh 10 dari 10 pemeriksaan berhasil lulus, sehingga implementasi program telah memenuhi kriteria yang ditentukan.

2) *SelfCheck*



Berikut merupakan tangkapan layar hasil pengujian *SelfCheck* yang menunjukkan bahwa 23 dari 23 pemeriksaan berhasil lulus tanpa kendala.

3) *TugasApp*



Berikut merupakan tangkapan layar hasil eksekusi *TugasApp* yang menampilkan *output* program, meliputi perhitungan total pembayaran beserta biaya administrasi, kebijakan pembayaran, serta pengiriman notifikasi status pesanan melalui beberapa kanal. Program berhasil dijalankan dan menghasilkan output sesuai dengan yang diharapkan.

D. *LINK REPOSITORY GITHUB*

Seluruh kode program pada Praktikum 6 yang telah diunggah ke *repository GitHub*.

Berikut merupakan *link* dari *repository PrakPBO_B_2511532002:*

https://github.com/haliyaismaa/PrakPBO_B_2511532002/tree/main/PrakPBO_B_2511532002

Untuk file kode program tugas dan juga laporan dari praktikum pekan ke 6 dapat diakses pada *path PrakPBO_B_2511532002/src/Praktikum6/Tugas/abstraction-2511532002*